Week1: Concept Check

Day1: Python Fundamentals & Data Structures

1. What's the difference between a list and a tuple?

Key Features	List	Tuples
Syntax	Defined using square brackets: [1, 2, 3]	Defined using parentheses: (1, 2, 3)
Mutability	Mutable (can be modified after creation: items can be changed, added, or removed)	Immutable (cannot be modified after creation)
Methods	.append(), .insert(), .pop(), .remove(), .sort(), etc.	.count(), .index()
Performance	Slightly slower due to dynamic memory allocation	Fast and more memory efficient
Use Case	Collections of items that may change over time or need sorting	Fixed collections of heterogeneous data (e.g., coordinates, database records)

2. Write a function that returns the square of a number.

```
def square_number(n):
    return n * n

result = square_number(5)
print(result)

#Output: 25
```

3. What does *args do in a function signature?

*args allows a function to accept a variable number of positional arguments. It collects any extra positional arguments passed to the function into a tuple.

```
def print_args(*args):
    print(args)

print_args(1, 2, 3)
```

4. What's the output of a basic for loop over range(5)?

The range(5) function generates numbers starting from 0 up to (exclude) 5. Iterating over it prints each number on a new line:

```
Output:
0
1
2
3
4
```

5. What is a dictionary comprehension?

A dictionary comprehension is a concise, readable syntax used to create a new dictionary from an iterable by specifying key-value expressions inside curly brackets {}.

```
# Creates a dictionary mapping numbers to their squares
squares = {x: x**2 for x in range(4)}
print(squares)

 # Output: {0: 0, 1: 1, 2: 4, 3: 9}
```

Day 2: NumPy Array Operations

1. What is broadcasting in NumPy?

Broadcasting in NumPy allows operations on arrays of different shapes by virtually stretching the smaller array to match the larger one, without duplicating data in memory.

NumPy compares shapes from right to left. Two dimensions are compatible if:

1. They are **equal**, or
2. One of them is **1**.

```
import numpy as np
arr = np.array([1, 2, 3])
result = arr + 10           # 10 is broadcasted
print(result)

 # Output: [11 12 13]
```

2. How do you select all rows where a column value > 10 in a 2D array?

You can achieve this using boolean indexing. By passing a condition on a specific column index as an index into the 2D array, NumPy filters and returns only the rows where that condition is True.

```
import numpy as np
arr = np.array([[5, 12], [15, 8], [3, 20]])
filtered_rows = arr[arr[:, 1] > 10]
print(filtered_rows)
# Output: [[ 5 12], [ 3 20]]
```

3. Difference between .reshape() and .flatten()?

.reshape(shape): Changes the shape of an array without changing its data. It returns a view of the original array whenever possible. The total number of elements must remain the same.
.flatten(): Always collapses any multi-dimensional array into a 1D array and strictly returns a copy of the data. Modifications made to a flattened array will not affect the original array.

4. What does axis=0 vs axis=1 mean in np.sum()?

axis=0 (Rows / Downwards): Performs the operation vertically down the columns. For a 2D array, it sums the values of all rows for each column, resulting in a single row output.
axis=1 (Columns / Across): Performs the operation horizontally across the rows. For a 2D array, it sums the values of all columns for each row, resulting in a single column output.

5. How do you generate a random array of shape (5,5)?

Use np.random.rand() or np.random.randn():

```
import numpy as np

# Uniform random floats between 0 and 1
arr = np.random.rand(5, 5)

# Or random integers between 0 and 10:
# arr = np.random.randint(0, 10, size=(5, 5))
```

Day 3: Pandas DataFrames & Series

1. Difference between .loc[] and .iloc[]?

.loc[] (Label-based): Used to select rows and columns by their labels or names (e.g., column names like 'Age' or index labels). Note that the end-point of a slice is inclusive.
iloc[] (Integer position-based): Used to select rows and columns strictly by their integer index positions (0 indexed). The end-point of a slice is exclusive.

2. How do you filter rows where two conditions are both true?

We can combine the conditions using the bitwise AND operator (&), and each condition must be wrapped in parentheses () due to operator precedence in pandas.

```
import pandas as pd
df = pd.DataFrame({'Age': [25, 30, 22, 45], 'Salary': [50000, 80000, 40000, 120000]})
filtered_df = df[(df['Age'] > 25) & (df['Salary'] > 60000)]
```

3. What does .groupby() return before you call an aggregation on it?

It returns a DataFrameGroupBy (or SeriesGroupBy) object. This is an intermediate, lazy-evaluated object that holds the groups internally but hasn't computed anything yet until an aggregation function (like .mean(), .sum(), or .count()) is applied.

4. How do you check for and count missing values in a DataFrame?

We can combine .isna() (or .isnull()) with .sum() to get a count of missing values per column or across the entire DataFrame.

```
# Count  missing values per column
print(df.isnull().sum())

# Total missing values in DataFrame
print(df.isnull().sum().sum())
```

5. What's the difference between df.copy() and just assigning df2 = df?

df2 = df (Assignment): Creates a reference to the exact same DataFrame object in memory. Modifying df2 will also modify the original df.
df.copy() (Deep Copy): Creates a completely independent duplicate of the DataFrame in memory. Modifications made to the copy will not affect the original.

Day 4: Pandas DataFrames & Series

1. **What is the difference between .drop() and .dropna()?**

   .drop(): Removes specific rows or columns by explicitly providing their labels or indices

   ```
   # Remove specific row by label/index
   df.drop(index=0)
   ```

   .dropna(): removes rows or columns that contain missing values based on the presence of missing values (NaN or None).

   ```
   # Remove row containing any missing values
   df.dropna()
   ```

2. How do you change the dtype of a column — and when would you need to?

We can change a column's data type using the .astype() method or conversion functions like pd.to_numeric().

When needed: Numeric values imported as strings, dates/timestamps parsed as plain strings, or reducing memory usage by downcasting types.

```
import pandas as pd
df['column_name'] = df['column_name'].astype('int')
df['price'] = pd.to_numeric(df['price'], errors='coerce')
```

3. What does .apply() do and how is it different from vectorized operations?

.apply(): Passes a custom Python function along an axis of a DataFrame or Series, looping through elements in Python under the hood.

Vectorized operations: Run directly on entire arrays in optimized C/Fortran code via NumPy, executing significantly faster.

4. Difference between .pivot() and .pivot_table()?

.pivot(): Reshapes data based on unique index/column pairs. It cannot handle duplicate entries and will throw a ValueError if duplicates exist.

.pivot_table(): Similar to .pivot(), but includes an aggregation function (aggfunc) to automatically handle and aggregate duplicate entries. The default aggregation function is mean.

5. What does .merge() do and what are the 4 types of joins?

.merge() combines two pandas DataFrames based on common columns or indices, similar to an SQL database join.

There are 4 types of joins:

1. **inner (Default):** Keeps only rows with keys matching **both** DataFrames (intersection).
2. **outer (Full Outer):** Keeps all rows from **both** DataFrames, filling missing matches with NaN.
3. **left (Left Outer):** Keeps all rows from the **left** DataFrame and matching rows from the right.
4. **right (Right Outer):** Keeps all rows from the **right** DataFrame and matching rows from the left.

Day 5: Data Visualization (Matplotlib & Seaborn)

1. When would you use a bar chart vs a histogram?

Bar Chart: Used for categorical data (discrete categories or groups). Bars represent distinct groups with spaces between them to emphasize categorical separation.
Histogram: Used for continuous numerical data. Groups data into contiguous numerical intervals or bins to show distribution frequency with no gaps.

2. What is the difference between plt.plot() and sns.lineplot()?

plt.plot() (Matplotlib): A low-level function that plots raw data points directly as passed, and it creates a messy zig-zag if an x-value has multiple y-values.
sns.lineplot() (Seaborn): A high-level statistical plotting function that automatically aggregates data, plots means, and calculates/shades confidence intervals.

3. What does a boxplot actually show — what are the whiskers?

A boxplot displays the five-number summary: minimum, Q1, median, Q3, and maximum.
• The Box: Represents the Interquartile Range (IQR) from Q1 (25th percentile) to Q3 (75th percentile), with a line for the median.
• The Whiskers: Extend to furthest data points within 1.5 × IQR from the quartiles. Points beyond are plotted as outliers.

4. How do you plot multiple charts in one figure using Matplotlib?

We can use plt.subplots() to create a figure container and a grid of axes, then plot data onto each specific axis object.

```
import matplotlib.pyplot as plt
fig, (ax1, ax2) = plt.subplots(nrows=1, ncols=2, figsize=(10, 4))
ax1.plot([1, 2, 3], [4, 5, 6])
ax2.scatter([1, 2, 3], [6, 5, 4])
plt.tight_layout()
plt.show()
```

5. What is a heatmap useful for and what kind of data does it work best with?

A **heatmap** uses color gradients to instantly reveal patterns, correlations, and anomalies in data.

It works best with **2D numerical matrices**, such as:

- **Correlation matrices** (showing relationships between variables)
- **Pivot tables** (comparing two categorical variables)
- **Confusion matrices** (evaluating machine learning models)